

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA1012	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	MANIYAM ANUSHKA	Reg. No.:	192525411
Date:	5-08-2026	Duration:	60 minutes

Code of Conduct

I MANIYAM ANUSHKA(192525411) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: MANIYAM ANUSHKA

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.

- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.

Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

Industry Context

Hospitals today face increasing patient volumes, limited medical resources, and growing demand for efficient healthcare delivery. Beds, equipment, staff, and appointments must be actively coordinated to maintain quality care while controlling costs. Traditional hospital information systems operate as isolated modules, making cross-department coordination difficult. Advances in Artificial Intelligence, IoT, and predictive analytics now allow hospitals to forecast admissions, track resources in real time, and optimize patient flow across emergency, outpatient, laboratory, and inpatient units, making AI-driven hospital resource management a critical part of modern digital healthcare ecosystems.

Problem Statement

Hospitals frequently experience overcrowding, prolonged waiting times, inefficient bed utilization, and uneven distribution of medical resources. Existing hospital information systems mainly manage administrative records but lack intelligent decision-support for optimizing patient flow. Delays in transfers, diagnostics, and discharge planning reduce effective hospital capacity, and administrators lack predictive insight into fluctuating demand. MediSync AI addresses this gap with an AI-powered platform that continuously monitors patient movement, predicts resource requirements, and recommends optimal allocation of beds, staff, and equipment.

1. Analysis of System Requirements

1.1 System Objectives

- **Real-time visibility:** continuously monitor bed, staff, and equipment availability across all departments.
- **Predictive intelligence:** forecast admissions, discharges, and resource demand using machine learning.
- **Workflow optimization:** optimize patient scheduling, admissions, and interdepartmental transfers to cut waiting times.
- **Decision support:** provide real-time dashboards and predictive analytics for administrators and clinical staff.

- **Operational efficiency:** improve resource utilization and reduce operational costs while sustaining care quality.

1.2 Functional Requirements

#	Functional Requirement
1	Monitor real-time availability and status of hospital beds, medical equipment, and healthcare staff across departments.
2	Predict patient admissions, discharge times, and resource demand using machine learning models trained on historical and real-time data.
3	Optimize scheduling of patient admissions, appointments, and interdepartmental transfers automatically.
4	Reduce patient waiting times through intelligent, workflow-driven task routing.
5	Provide real-time dashboards and alerts for hospital administrators and medical staff.
6	Generate predictive analytics reports to support emergency preparedness and capacity planning.
7	Integrate with existing Hospital Information Systems (HIS/EHR), IoT sensors, and Emergency Department systems.
8	Support role-based access for administrators, clinical staff, and patients.

1.3 Non-Functional Requirements

Quality Attribute	Non-Functional Requirement
Scalability	Must handle growing patient volumes and additional hospital branches without performance degradation.
Performance	Real-time dashboards must reflect bed/equipment status changes within 1-2 seconds; ML predictions returned within a few seconds.

Quality Attribute	Non-Functional Requirement
Reliability	System must guarantee no loss of critical events (admissions, discharges, emergency alerts) even during partial outages.
Availability	Target 99.9% uptime, since hospital operations run continuously (24x7).
Security & Compliance	Must protect patient health information (PHI) with encryption, access control, and audit logging in line with healthcare data protection regulations.
Maintainability	Individual modules must be independently updatable as clinical workflows or ML models evolve.
Interoperability	Must integrate with existing HIS/EHR systems and IoT medical devices using standard protocols/APIs.

1.4 Key Quality Attributes Influencing the Architecture

Scalability, performance, and reliability dominate the design because hospital demand fluctuates sharply (e.g., emergencies, seasonal surges) and downtime directly risks patient safety. Security is equally critical due to the sensitivity of patient health information, while maintainability matters because clinical workflows and ML models will keep evolving. Availability must approach 24x7 continuity since hospitals never fully close.

2. Selection of a Suitable Software Architecture

A Hybrid Architecture combining Microservices, an Event-Driven backbone, and a Layered presentation/gateway structure is selected for MediSync AI.

2.1 Justification

- **Independent, department-aligned modules:** Bed management, staff scheduling, equipment tracking, admissions, and notifications map naturally to independent bounded contexts, which microservices support directly — each can be developed, deployed, and scaled independently.

- **Real-time coordination needs:** Patient flow across departments is inherently event-based (a discharge triggers bed release, which triggers admission of the next patient). An event-driven backbone (Kafka/RabbitMQ) propagates these events asynchronously without tightly coupling services.
- **Heterogeneous, high-load AI workloads:** The ML Prediction Engine has very different scaling and resource needs (GPU/CPU-heavy, batch + streaming inference) than transactional services like Bed Management — separating it as its own service allows independent scaling.
- **Integration with legacy systems:** Hospitals already run HIS/EHR systems and IoT sensor networks. A layered API Gateway cleanly isolates these external integrations from internal services.
- **Incremental adoption:** A microservices approach lets the hospital roll the platform out department-by-department rather than requiring a single, risky big-bang deployment.

2.2 Why Not Alternatives

- **Pure Layered/Monolithic:** would tightly couple bed management, scheduling, and prediction logic, making independent scaling and department-wise deployment difficult, and risking system-wide outages from a single fault.
- **Pure Client-Server:** does not naturally support the asynchronous, high-frequency event flows (sensor updates, admission/discharge triggers) central to real-time patient flow.
- **Pure Serverless:** is attractive for bursty workloads but is less suited to the always-on, low-latency, stateful monitoring hospitals need for life-critical operations.

3. Software Architecture Diagram

The diagram below illustrates the six-layer hybrid architecture: presentation clients, the API gateway/security layer, the microservices layer (including the AI/ML Prediction Engine), the event bus, the data layer, and external/IoT systems.

MediSync AI — Hybrid Microservices + Event-Driven Architecture

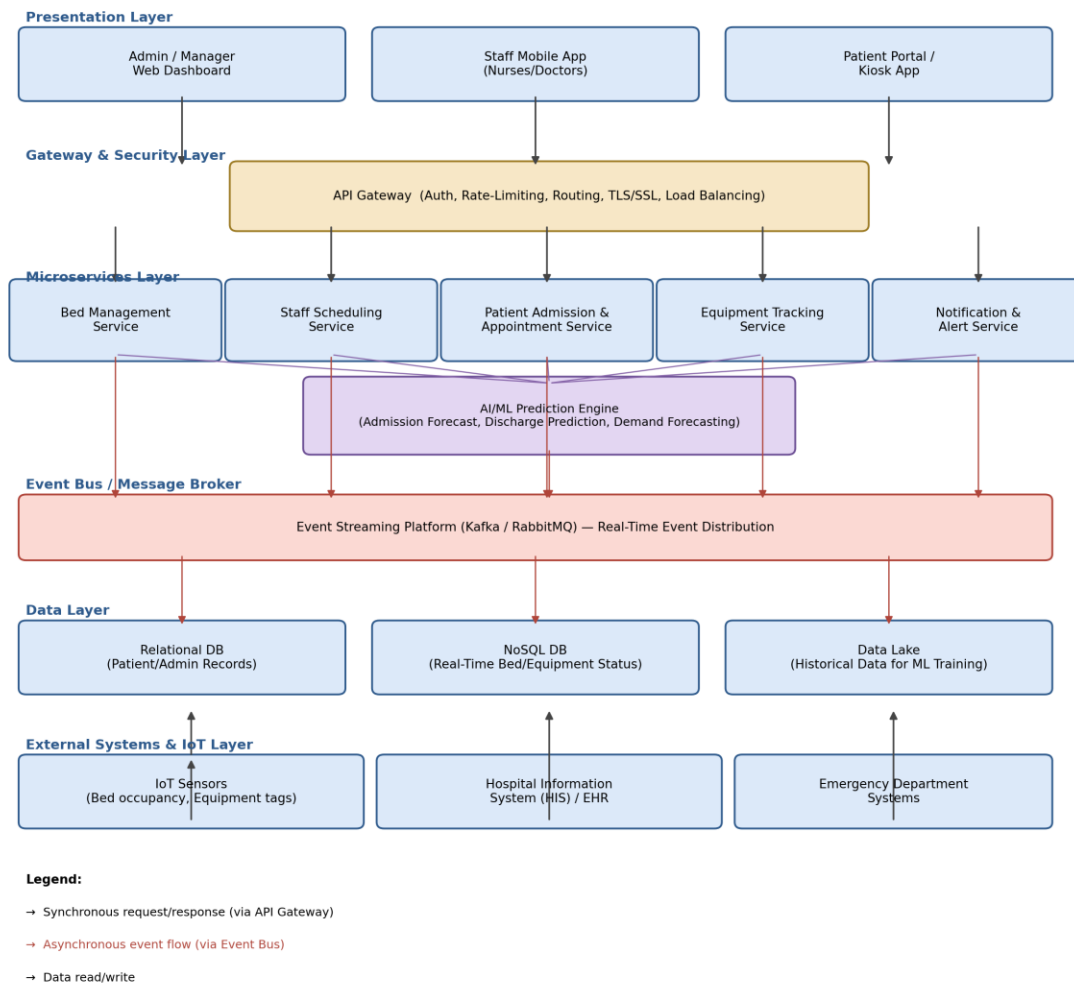


Figure 1: MediSync AI — Hybrid Microservices + Event-Driven Architecture

4. Components and Their Interactions

4.1 Component Responsibilities

- **Presentation Layer (Web Dashboard, Mobile App, Patient Portal):** provides role-specific interfaces for administrators, clinical staff, and patients to view resource status, schedules, and alerts.
- **API Gateway:** single entry point that handles authentication, authorization, rate-limiting, routing, and TLS termination, shielding internal services from direct external access.
- **Bed Management Service:** tracks real-time bed occupancy/availability and processes allocation and release requests.
- **Staff Scheduling Service:** manages healthcare personnel rosters, shift assignments, and availability.
- **Patient Admission & Appointment Service:** handles admissions, discharges, transfers, and appointment bookings.
- **Equipment Tracking Service:** monitors location and availability of shared medical equipment via IoT tags.
- **Notification & Alert Service:** pushes real-time alerts (e.g., bed shortage, overdue discharge) to relevant staff and dashboards.
- **AI/ML Prediction Engine:** consumes historical and streaming data to forecast admissions, discharge timing, and resource demand, publishing predictions back onto the event bus.
- **Event Streaming Platform (Kafka/RabbitMQ):** decouples services by distributing events (e.g., "bed released", "patient admitted") asynchronously to all interested subscribers.
- **Data Layer:** a relational database for structured patient/admin records, a NoSQL store for high-frequency real-time status data, and a data lake for historical data used in ML training.
- **External Systems (IoT sensors, HIS/EHR, Emergency Department systems):** feed real-world sensor and legacy-system data into the platform through the event bus and gateway.

4.2 Communication and Workflow

Clients communicate synchronously with backend services through the API Gateway using REST/HTTPS calls, which is appropriate for user-initiated actions such as viewing a dashboard or booking an appointment. Internally, when a state-changing action occurs — for instance, a patient is discharged — the owning microservice publishes an event to the message broker rather than calling

other services directly. Interested services (Bed Management, Notification Service, the Prediction Engine) subscribe to relevant event topics and react independently, which is what allows the system to add new consumers later without changing existing services. The AI/ML Prediction Engine continuously ingests both real-time events and historical data from the data lake, and periodically publishes predictions (e.g., expected admissions in the next 4 hours) back onto the event bus for dashboards and the Notification Service to consume. This event-driven workflow keeps end-to-end patient-flow coordination near real-time while keeping every service independently deployable.

5. Discussion of Quality Attributes

Quality Attribute	How the Architecture Addresses It
Scalability	The microservices layer allows each service (Bed Management, Staff Scheduling, Prediction Engine, etc.) to be scaled independently based on load. During peak admission periods, the Patient Admission Service and Prediction Engine can be horizontally scaled without affecting other services. The event bus decouples producers and consumers, so new hospital departments or IoT sources can be added without redesigning the system.
Security	The API Gateway centralizes authentication (OAuth2/JWT), authorization, and TLS termination, so no client ever talks to a microservice directly. Role-based access control restricts dashboards for administrators, staff and patients. All patient data (PHI) in the relational database is encrypted at rest and in transit to comply with healthcare data regulations (e.g., HIPAA-equivalent standards).
Performance	Real-time bed and equipment status is served from a low-latency NoSQL store, while heavier historical queries hit the data lake, keeping the operational path fast. The event-driven bus enables near real-time updates (sub-second) to dashboards without polling,

Quality Attribute	How the Architecture Addresses It
	and the ML Prediction Engine runs asynchronously so predictions never block request/response paths.
Reliability	Services are loosely coupled through the message broker (Kafka/RabbitMQ), so a failure in one microservice (e.g., Notification Service) does not cascade to others. Message queues buffer events during downtime, guaranteeing eventual delivery. Each service can be deployed with redundant instances behind the gateway for failover.
Maintainability	Each microservice owns a bounded context (single responsibility), so teams can update, test, and redeploy the Equipment Tracking Service, for example, without touching Bed Management. Clear API contracts at the gateway and well-defined event schemas on the bus reduce ripple effects when features are added.
Availability	Containerized microservices (Docker/Kubernetes) support rolling updates and auto-restart of failed instances with zero downtime. Data replication across the relational DB, NoSQL store, and data lake ensures no single point of failure. Load balancing at the API Gateway routes traffic away from unhealthy instances.

6. Justification of Architectural Decisions

6.1 Rationale

The hybrid microservices and event-driven approach was chosen because hospital resource management is naturally composed of independent, department-scoped functions (beds, staff, equipment, admissions) that must nonetheless react to shared, real-time events (a discharge affects bed availability, staffing, and the next admission simultaneously). Microservices give each function independent ownership and

scaling, while the event bus provides the real-time coordination a purely request/response (client-server) design cannot offer as cleanly.

6.2 Trade-offs Considered

- **Complexity vs. flexibility:** microservices and event-driven messaging introduce more operational complexity (service discovery, distributed monitoring, eventual consistency) than a monolith, but this is accepted in exchange for independent scalability and fault isolation, which matter more in a life-critical, always-on hospital setting.
- **Eventual consistency vs. strict consistency:** resource states (e.g., bed status) propagate via events and may be briefly stale across services; this is acceptable for dashboards/alerts but critical transactional actions (e.g., confirming an admission) still go through synchronous, gateway-mediated calls to the owning service.
- **Operational overhead vs. resilience:** running a message broker and container orchestration platform requires more DevOps investment than a single deployable unit, but this is offset by higher fault tolerance and the ability to patch/update one service without hospital-wide downtime.

6.3 Support for Future Enhancements and Long-Term Maintainability

- **System integration:** new hospital branches, additional IoT device types, or third-party EHR systems can be onboarded by adding new event producers/consumers without altering existing services.
 - **Feature growth:** new capabilities (e.g., ambulance routing, telemedicine scheduling) can be added as new microservices that subscribe to existing events, avoiding disruptive rewrites.
 - **ML model evolution:** the AI/ML Prediction Engine can be retrained, A/B tested, or replaced independently since it interacts with the rest of the system only through well-defined events and APIs.
 - **Long-term maintainability:** clear service boundaries, versioned API contracts at the gateway, and well-defined event schemas keep the codebase modular, making it easier for future teams to extend MediSync AI as hospital needs evolve.
-